

(SOLUTION)

CPCS-302-BE

Final Lab Exam
(Fall 2022)

Date: 09-12-2021

ID: _____

Name: _____

Q. No. 1: Find LR(1) items of Canonical LR parser for the following augmented grammar:

(3)

- $S' \rightarrow S$
1. $S \rightarrow AB$
2. $A \rightarrow aBb$
3. $A \rightarrow a$
4. $B \rightarrow d$

Solution:

I_0 :

$S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot AB, \$$
 $A \rightarrow \cdot aBb, d$
 $A \rightarrow \cdot a, d$
($\checkmark S, \checkmark A, \checkmark d$)

I_1 :

$S' \rightarrow S \cdot, \$$
 $\textcircled{0} \xrightarrow{S} \textcircled{1}$

I_2 :

$S \rightarrow A \cdot B, \$$
 $B \rightarrow \cdot d, \$$
 $\textcircled{0} \xrightarrow{A} \textcircled{2} \quad (\checkmark B, \checkmark d)$

I_3 :

$A \rightarrow a \cdot Bb, d$
 $A \rightarrow a \cdot, d$
 $B \rightarrow \cdot d, b$
 $\textcircled{0} \xrightarrow{a} \textcircled{3}$

I_4 :

$S \rightarrow AB \cdot, \$$
 $\textcircled{2} \xrightarrow{B} \textcircled{4}$

I_5 :

$B \rightarrow d \cdot, \$$
 $\textcircled{2} \xrightarrow{d} \textcircled{5}$

I_6 :

$A \rightarrow aB \cdot b, d$
 $\textcircled{3} \xrightarrow{B} \textcircled{6} \quad (\checkmark b)$

I_7 :

$B \rightarrow d \cdot, b$
 $\textcircled{3} \xrightarrow{d} \textcircled{7}$

I_8 :

$A \rightarrow aBb \cdot, d$
 $\textcircled{6} \xrightarrow{b} \textcircled{8}$

Question B3: Consider the following Context Free Grammar

[3 Marks]

$$\begin{aligned} A &\rightarrow A * B \mid B \\ B &\rightarrow B \wedge C \mid C \\ C &\rightarrow (A) \mid a \mid b \mid c \mid 2 \mid 4 \end{aligned}$$

where '*', '^', '(', ')', 'a', 'b', 'c', '2' and '4' are terminals, and 'A', 'B' and 'C' are non-terminals. Parse the sentence $(a \wedge b) + 2$ using left-most derivation.

Solution:

$$\begin{aligned} A &\Rightarrow \dot{A} + B \\ &\Rightarrow \dot{B} + B \\ &\Rightarrow \dot{C} + B \\ &\Rightarrow (\dot{A}) + B \\ &\Rightarrow (\dot{B}) + B \\ &\Rightarrow (\dot{B} \wedge C) + B \\ &\Rightarrow (\dot{C} \wedge C) + B \\ &\Rightarrow (a \wedge \dot{C}) + B \Rightarrow (a \wedge b) + \dot{B} \Rightarrow (a \wedge b) + \dot{C} \Rightarrow (a \wedge b) + 2 \end{aligned}$$

Question B4: Consider the following CFG grammar over the non-terminals $\{S, A, B, D\}$ and terminals $\{a, b, d, e\}$ with the productions below and start symbol S.

[6 Marks]

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow BdA \mid aA \mid C \\ B &\rightarrow bC \\ C &\rightarrow eC \mid E \end{aligned}$$

a) Compute FIRST and FOLLOW sets of every non-terminal of this CFG.

FIRST / FOLLOW Table		
Non terminal	FIRST	FOLLOW
S	a	\$
A	a, b, e	\$
B	b	d
C	e, E	d

b) Construct non-recursive predictive parse table for this grammar.

Non terminal	a	b	d	e	\$
S	$S \rightarrow aA$				
A	$A \rightarrow aA$	$A \rightarrow BdA$			
B		$B \rightarrow bC$			$A \rightarrow E$
C			$C \rightarrow E$	$C \rightarrow eC$	

Q. No. 4: Find FIRST and FOLLOW sets for the following grammar and then construct non-recursive predictive parse table.

SO: 1 CLO: 10 Total Points: 20 Obtained Points: 19

$S \rightarrow ACB \mid CbB \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \epsilon$

$C \rightarrow h \mid \epsilon$

Solution:

FIRST(S) = { d, g, h, b, a, ϵ }

FIRST(A) = { d, g, h, ϵ }

FIRST(B) = { g, ϵ }

FIRST(C) = { h, ϵ }

FOLLOW(S) = { \$ }

FOLLOW(A) = { g, h, \$ }

FOLLOW(B) = { \$, a, g, h }

FOLLOW(C) = { g, h, \$, b, a }

	a	b	d	g	h	\$
S	$S \rightarrow Ba$	$S \rightarrow CbB$	$S \rightarrow ACB$	$S \rightarrow ACB$	$S \rightarrow ACB$	$S \rightarrow ACB$
A			$A \rightarrow da$	$A \rightarrow BC$	$A \rightarrow BC$	
B	$B \rightarrow \epsilon$			$B \rightarrow g$	$B \rightarrow \epsilon$	
C	$C \rightarrow \epsilon$	$C \rightarrow h$		$C \rightarrow \epsilon$	$C \rightarrow h$	

Q. No. 2: Find FIRST and FOLLOW sets for the following grammar and then construct top-down non-recursive Predictive Parse Table for it.

SO: 1 CLO: 10 Total Points: 3 Obtained Points: 2.5

$$\begin{aligned} S &\rightarrow aBA \mid b \\ A &\rightarrow SaB \mid Bb \mid \epsilon \\ B &\rightarrow cB \mid Sd \mid \epsilon \end{aligned}$$

Solution:

$$\begin{aligned} \text{First}(S) &= \{a, b\} \\ \text{First}(A) &= \{a, b, c, \text{null}\} \\ \text{First}(B) &= \{c, a, b, \text{null}\} \end{aligned}$$

$$\begin{aligned} \text{Follow}(S) &= \{\$, a, d\} \\ \text{Follow}(A) &= \{\$, a, d\} \\ \text{Follow}(B) &= \{\$, a, d, b\} \end{aligned}$$

Parse Table:

Non-Terminals	a	b	c	d	S
S	$S \rightarrow aBA$ Sync	$S \rightarrow b$		Sync	Sync
A	$S \rightarrow aBA$ $A \rightarrow \epsilon$	$S \rightarrow b$	$B \rightarrow cB$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B	$S \rightarrow aBA$ Sync	$S \rightarrow b$	$B \rightarrow cB$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$

Q. No. 5: Consider the following grammar and its parse table.

SO: 1 CLO: 11 Total Points: 20 Obtained Points: 20

$S \rightarrow AC$
 $A \rightarrow aBCd \mid BD \mid \epsilon$
 $B \rightarrow bB \mid d$
 $C \rightarrow c \mid \epsilon$
 $D \rightarrow q$

	a	b	c	d	q	\$
S	$S \rightarrow AC$	$S \rightarrow AC$	$S \rightarrow AC$	$S \rightarrow AC$		$S \rightarrow AC$
A	$A \rightarrow aBCd$	$A \rightarrow BD$	$A \rightarrow \epsilon$	$A \rightarrow BD$		$A \rightarrow \epsilon$
B		$B \rightarrow bB$		$B \rightarrow d$		
C			$C \rightarrow c$	$C \rightarrow \epsilon$		$C \rightarrow \epsilon$
D					$D \rightarrow q$	

Show the actions of non-recursive predictive parser on the input "abddc\$".

Solution:

Stack	Input	Output
\$S	<u>a</u> b b d d c \$	$S \rightarrow AC$
\$CA	a b b d d c \$	$A \rightarrow aBCd$
\$CdCBa	a b b d d c \$	
\$CdCB	b b d d c \$	$B \rightarrow bB$
\$CdCBb	b b d d c \$	
\$CdCB	b d d c \$	$B \rightarrow bB$
\$CdCBb	b d d c \$	
\$CdCB	d d c \$	$B \rightarrow d$
\$CdCd	d d c \$	
\$CdC	d c \$	$C \rightarrow \epsilon$

Question 09: Consider the following grammar

(8 points)

$S \rightarrow XY \mid YX$
 $X \rightarrow \text{num} \mid \text{id}$
 $Y \rightarrow (Z) \mid \epsilon$
 $Z \rightarrow S + Z \mid \epsilon$

The non-recursive predictive parse table for this grammar is as follows:

	num	id	(	)	+	\$
S	$S \rightarrow XY$	$S \rightarrow YX$	$S \rightarrow YX$		$S \rightarrow YX$	$S \rightarrow YX$
X	$X \rightarrow \text{num}$	$X \rightarrow \text{id}$				
Y	$Y \rightarrow \epsilon$	$Y \rightarrow \epsilon$	$Y \rightarrow (Z)$		$Y \rightarrow \epsilon$	$Y \rightarrow \epsilon$
Z	$Z \rightarrow S + Z$	$Z \rightarrow S + Z$	$Z \rightarrow S + Z$	$Z \rightarrow S + Z$	$Z \rightarrow \epsilon$	$Z \rightarrow \epsilon$

Show the actions of non-recursive predictive parser on the input "(num +) id \$".

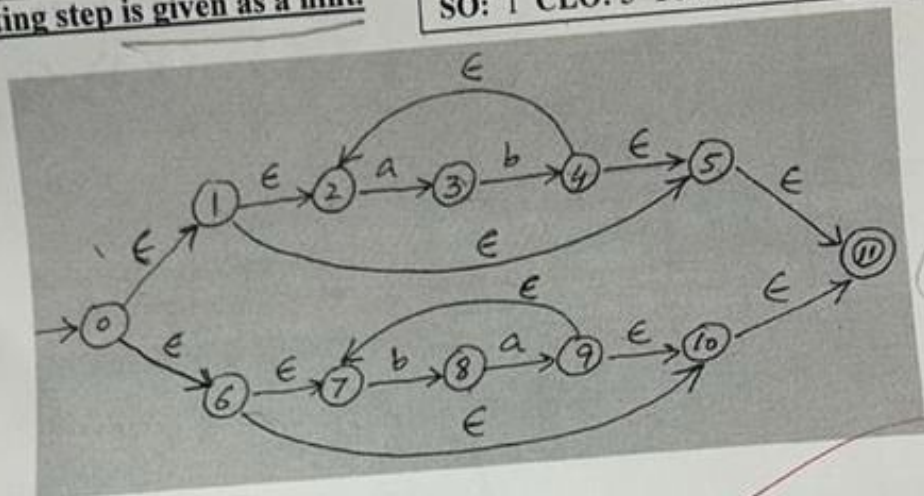
Stack	Input Buffer	Parser Actions
\$S	(num +) id \$	$S \rightarrow YX$
\$XY	(num +) id \$	$Y \rightarrow (Z)$
\$X)Z(	(num +) id \$	
\$X)Z	num +) id \$	$Z \rightarrow S + Z$
\$X)Z + S	num +) id \$	$S \rightarrow XY$
\$X)Z + YX	num +) id \$	$X \rightarrow \text{num}$
\$X)Z + Ynum	num +) id \$	
\$X)Z + Y	+) id \$	$Y \rightarrow \epsilon$
\$X)Z +	+) id \$	
\$X)Z	) id \$	$Z \rightarrow \epsilon$
\$X)	) id \$	
\$X	id \$	$X \rightarrow \text{id}$
\$id	id \$	
\$	\$	PARSING SUCCESSFULLY HALTS

2 3

Label it with ω
Exercise 8

Q. No. 3: Convert the following NFA into its equivalent DFA, by showing all steps in detail. Without detailed calculations, you will get 0 marks.
Note: Starting step is given as a hint.

SO: 1 CLO: 5 Total Points: 3 Obtained Points: 3



Solution:

$$\epsilon\text{-Closure}(\{0\}) = \{0, 1, 2, 5, 6, 7, 10, 11\} = A$$

$$\epsilon\text{-closure}(\text{move}(A, a)) = \{3, 9\} \quad \# \text{ don't null closure from this 3, include 9}$$

$$\{3, 9\} \Rightarrow \{3, 7, 9, 10, 11\} = B$$

$$\epsilon\text{-closure}(\text{move}(A, b)) = \{4, 8\} \quad \# \text{ don't null closure from this 4, include 8}$$

$$\{4, 8\} \Rightarrow \{2, 4, 5, 8, 11\} = C$$

$$\epsilon\text{-closure}(\text{move}(B, a)) = \{3\} = D$$

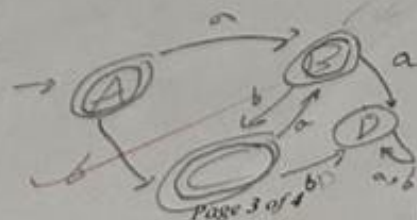
$$\epsilon\text{-closure}(\text{move}(B, b)) = \{4, 8\} = C$$

$$\epsilon\text{-closure}(\text{move}(C, a)) = \{3, 9\} = B$$

$$\epsilon\text{-closure}(\text{move}(C, b)) = \{4\} = D$$

$$\epsilon\text{-closure}(\text{move}(D, a)) = D$$

$$\epsilon\text{-closure}(\text{move}(D, b)) = D$$



①

$$S \rightarrow S + A \mid A$$

$$A \rightarrow A \mid B \mid B \mid \epsilon$$

$$B \rightarrow (S) \mid a$$

left recursive

②

$$S \rightarrow a b S A \mid S A a b \mid A c d \mid S a d$$

$$A \rightarrow A a \mid A b \mid a \mid b \mid \epsilon$$

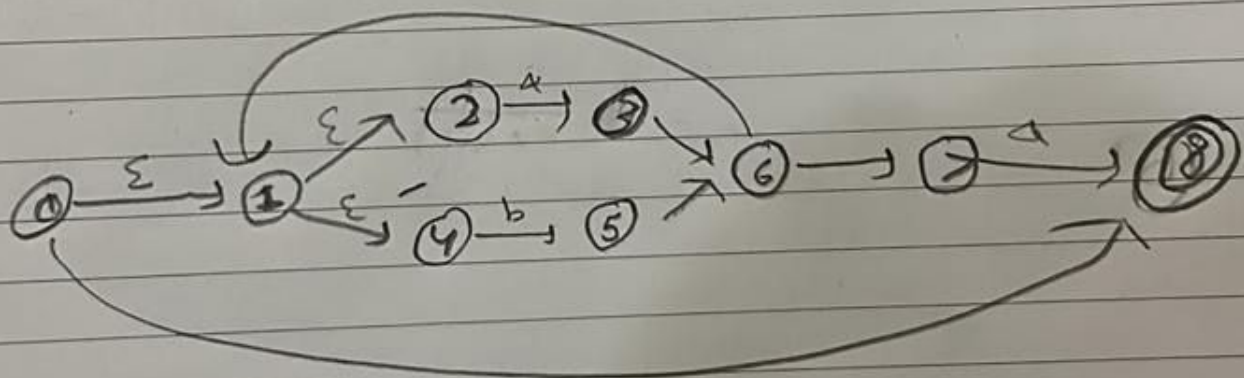
$$S \rightarrow a B A \mid b$$

Find First and Follow

$$A \rightarrow S a b \mid B b \mid \epsilon$$

والاجزاء مع السكك

$$B \rightarrow c S \mid B b \mid \epsilon$$

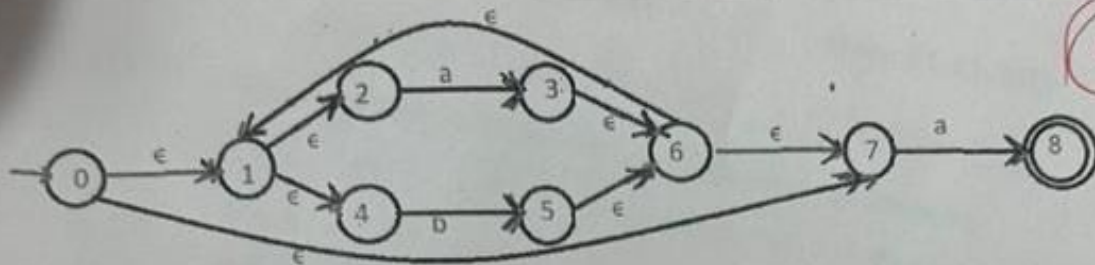


No. 3: Convert the following NFA into its equivalent DFA, by showing all steps in detail. Without detailed calculations, you will get 0 marks.

Note: Starting step is given as a hint.

SO: 1 CLO: 5 Total Points: 3 Obtained Points:

0.5



Solution:

$$\epsilon\text{-Closure}(\{0\}) = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$$

$\} = A$

$$(A, a) = \{3\} \leftarrow \epsilon\text{-closure}(\{1, 2, 3, 6, 7, 8\}) = B$$

$$(A, b) = \{5\} \leftarrow \epsilon\text{-closure}(\{1, 4, 5, 6, 7, 8\}) = C$$

$$(B, a) = \{3, 8\} = B$$

$$(B, b) = \{\} = D$$

$$(C, a) = \{8\} = E$$

$$(C, b) = \{5\} = C$$

$$(D, a) = \{8\} = E$$

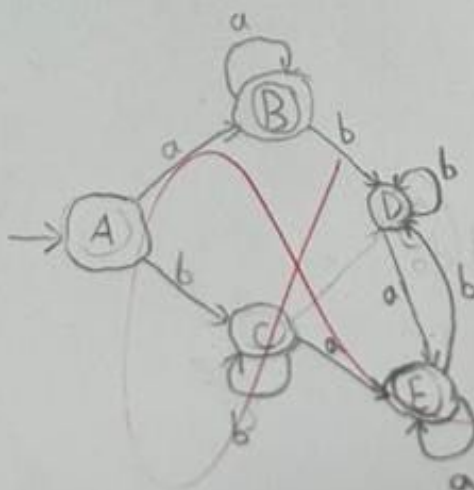
$$(D, b) = \{\} = D$$

$$(E, a) = \{8\} = E$$

$$(E, b) = \{\} = D$$

State Table for DFA:

	a	b
- A	B	C
+ B	B	D
+ C	E	C
+ D	E	D
+ E	E	D



Q. No. 3: Consider the following context-free grammar.

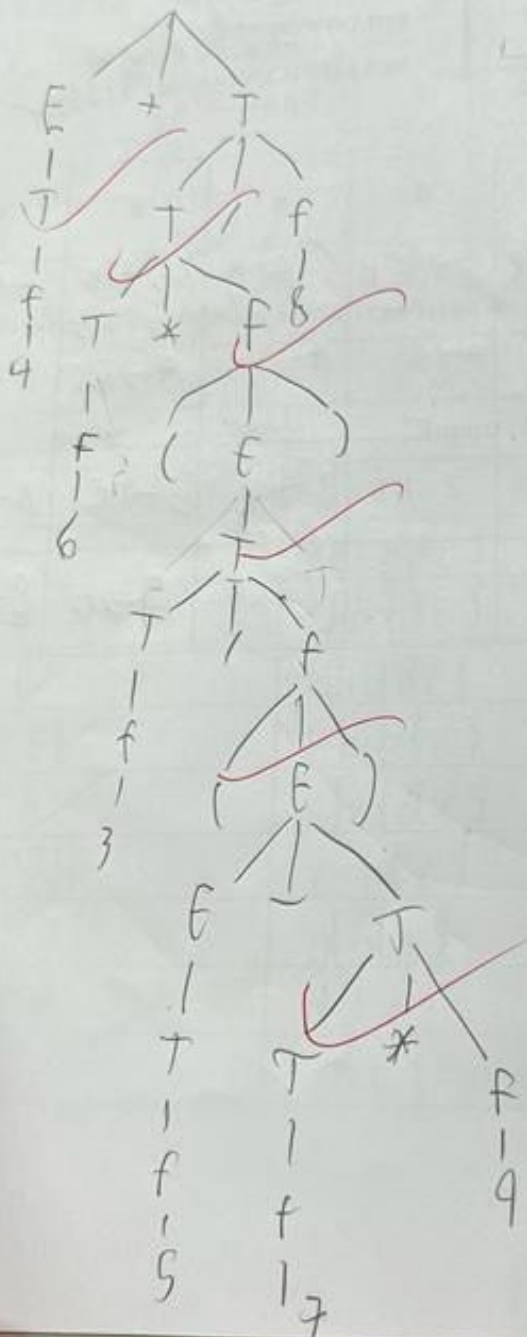
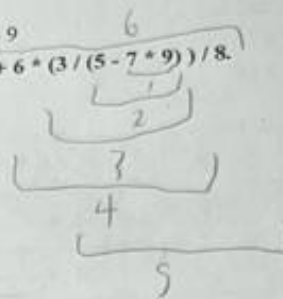
$$E \longrightarrow E + T \mid E \cdot T \mid T$$
$$T \longrightarrow T * F \mid T / F \mid F$$
$$T \rightarrow T \cdot F \mid T / F \mid F$$

$$F \rightarrow (E) \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Construct a parse tree for the expression $4 + 6 * (3 / (5 - 7 * 9)) / 8$.

Solution:

SO: 1 CLO: 7 Total Points: 20 Obtained Points: 20



Q. No. 4: Consider the following context-free grammar.

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

SO: 1 CLO: 7 Total Points: 20 Obtained Points:

- (i) Write semantic actions using Syntax Directed Translation Scheme to convert infix expressions into their equivalent postfix expressions.

Solution:

Productions

$$E \rightarrow E + T$$

$$E \rightarrow E - T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow T / F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow 0$$

$$F \rightarrow 1$$

$$F \rightarrow 2$$

.....

$$F \rightarrow 9$$

Semantic Actions

$$E \rightarrow E + T \quad \{ \text{print}(' + ') \}$$

$$E \rightarrow E - T \quad \{ \text{print}(' - ') \}$$

$$E \rightarrow T$$

$$T \rightarrow T * F \quad \{ \text{print}(' * ') \}$$

$$T \rightarrow T / F \quad \{ \text{print}(' / ') \}$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow 0 \quad \{ \text{print}('0') \}$$

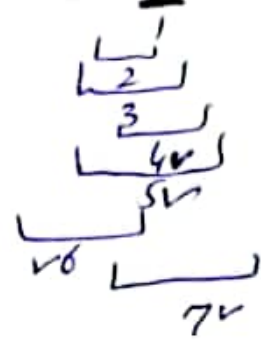
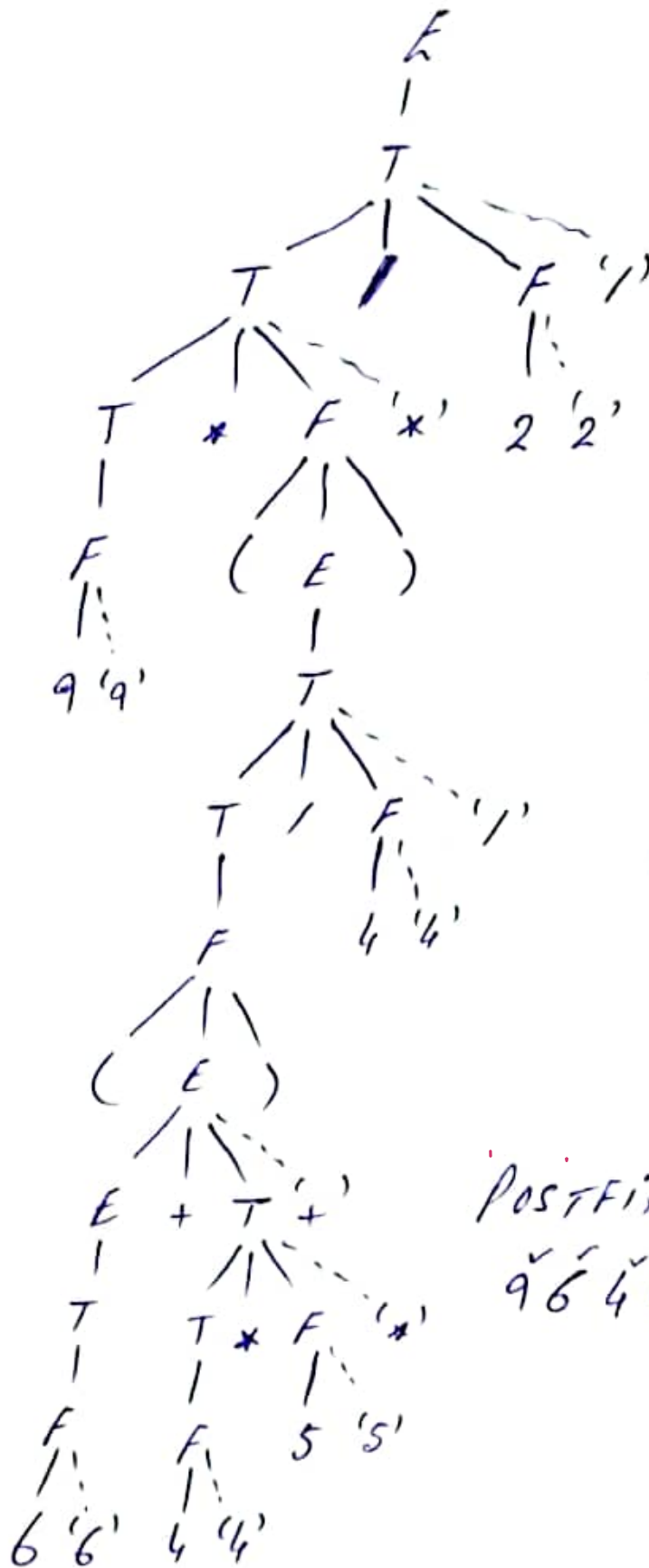
$$F \rightarrow 1 \quad \{ \text{print}('1') \}$$

$$F \rightarrow 2 \quad \{ \text{print}('2') \}$$

$$F \rightarrow 9 \quad \{ \text{print}('9') \}$$

- (ii) Construct annotated parse tree for converting infix expression $9 * ((6 + 4 * 5) / 4) / 2$ into its equivalent postfix form.

Solution:



POSTFIX:

9 6 4 5 * + 4 / * 2

Q. No. 3: Consider the following context-free grammar.

$$E \rightarrow E + T \mid E - T \mid T$$

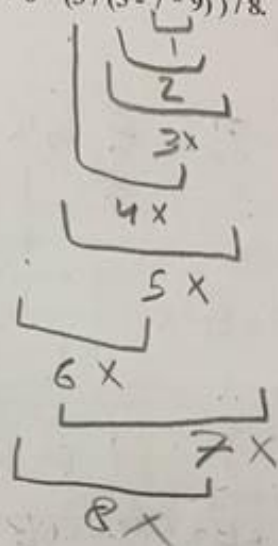
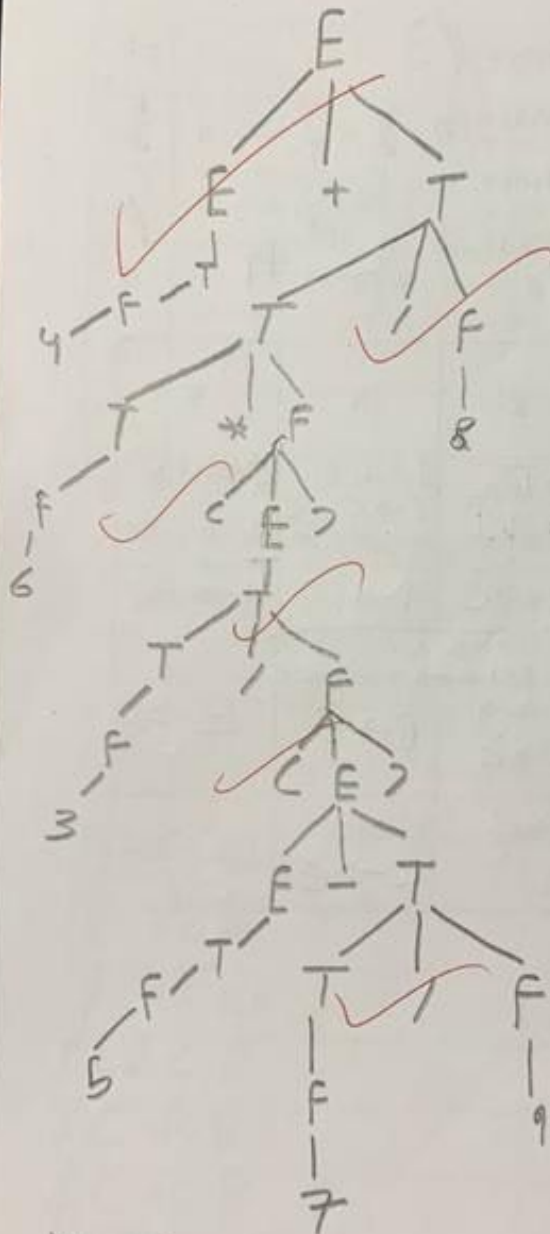
$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

SO: 1 CLO: 7 Total Points: 20 Obtained Points: 20

Construct a parse tree for the expression $4 + 6 * (3 / (5 - 7 * 9)) / 8$.

Solution:

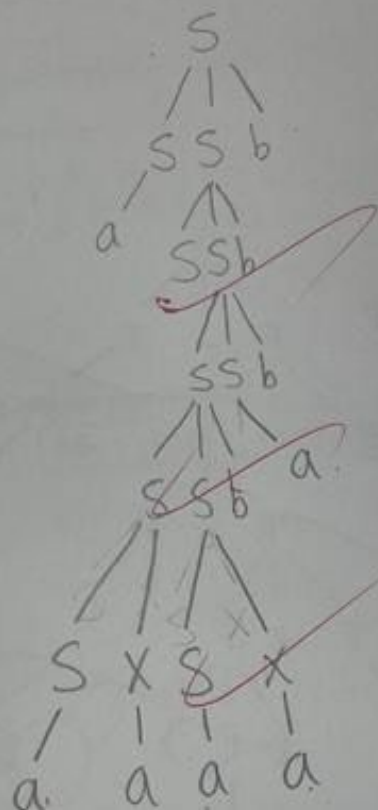


$$4 + 6 * (3 / (5 - 7 * 9)) / 8$$

Yield:

$$4 + 6 * (3 / (5 - 7 * 9)) / 8$$

Construct two different parse trees for the word 'aaaaababbb' to prove that it is ambiguous grammar.



Q. No. 2: Consider $\Sigma = \{a, b\}$. Write regular expressions for the following languages.

SO: 1 CLO: 3 Total Points: 10 Obtained Points: 10

(i) Language of all words that start with letter 'a' but have odd length.
Solution:

$$a[(a+b)(a+b)]^{\#}$$

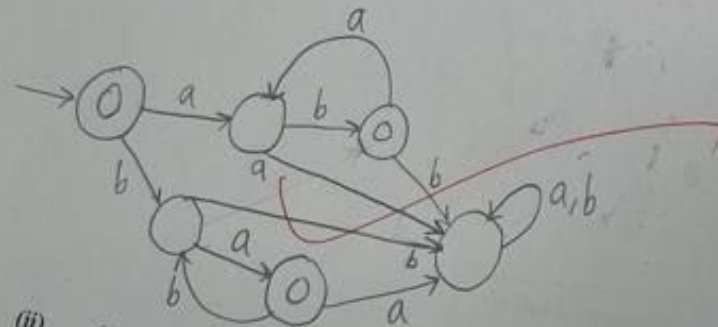
(ii) Language of all words which contain odd number of a's, and any number of b's (may be no b's at all).
Solution:

$$(b^{\#}ab^{\#}ab^{\#})^{\#}b^{\#}ab^{\#}$$

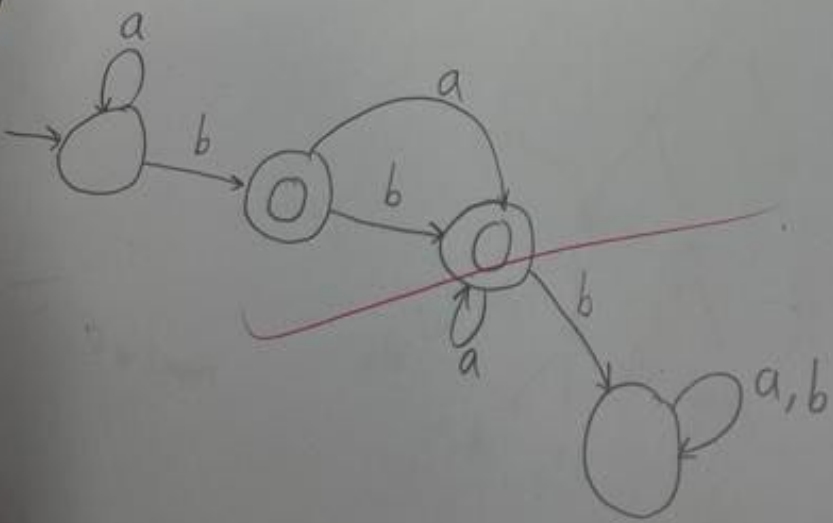
Q. No. 3: Construct DFA for the following regular expressions:

(i) $(ab)^{\#} + (ba)^{\#}$
Solution:

SO: 1 CLO: 5 Total Points: 10 Obtained Points: 10



(ii) $a^{\#}(b+bb)a^{\#}$
Solution:



Q. No. 1: What will be the output of the first three phases of compiler when the following assignment statement passes through them.
 $X = A * X + 0.5 * X * B$ (Here B is integer, while other variables are of type 'real')

SO: 1 CLO: 1 Total Points: 20 Obtained Points: 7

Solution:

$X = A * X + 0.5 * X * B$

Lexical Analyzer

$A \rightarrow \text{Real}$

$* \rightarrow \text{Mult}$

$X \rightarrow \text{Real}$

$+$ $\rightarrow$ Plus

$0.5 \rightarrow \text{Float}$

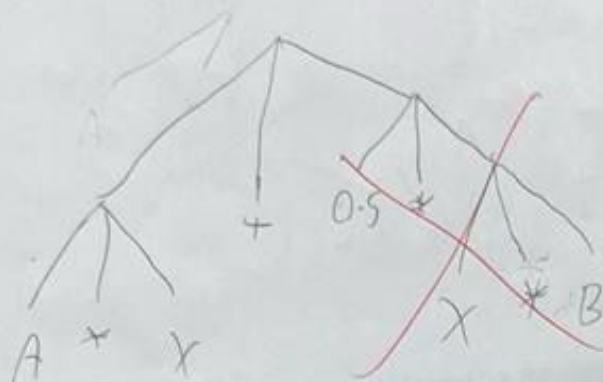
$* \rightarrow \text{Mult}$

$X \rightarrow \text{Real}$

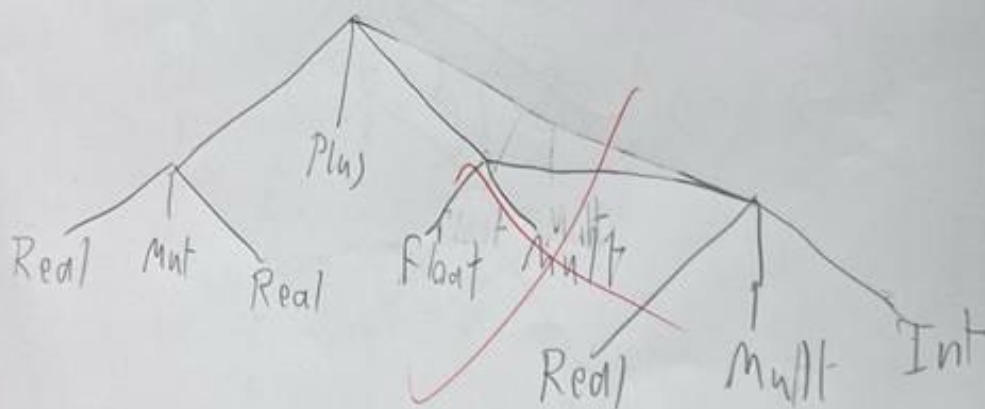
$* \rightarrow \text{Mult}$

$B \rightarrow \text{Int}$

Syntax Analyzer



Semantic Analyzer



ID: [REDACTED]

Name: [REDACTED]

AY/A

Section: CS3

Q. No. 1: In order to remove ambiguity, eliminate left recursion from the following grammars.

SO: 1 CLO: 4 Total Points: 4 Obtained Points: 4

- (i) $S \rightarrow S + A \mid A$
 $A \rightarrow A d B \mid B \mid \epsilon$
 $B \rightarrow (S) \mid id$

Solution:

$$\begin{aligned} S &\rightarrow A S' \mid S' \\ S' &\rightarrow + A S' \mid \epsilon \\ A &\rightarrow B A' \mid A' \\ A' &\rightarrow d B A' \mid null \\ B &\rightarrow (S) \mid id \end{aligned}$$

- (ii) $S \rightarrow abSA \mid SAab \mid Acd \mid Sad$
 $A \rightarrow Aa \mid Ab \mid a \mid b \mid \epsilon$

Solution:

$$\begin{aligned} S &\rightarrow abSA S' \mid Acd S' \mid S' \\ S' &\rightarrow Aab S' \mid ad S' \mid \epsilon \\ A &\rightarrow a A' \mid b A' \mid A' \\ A' &\rightarrow a A' \mid b A' \mid null \end{aligned}$$

Q. No. 2: Removing Left Recursion from the following grammars to make them LL(1).

SO: 1 CLO: 4 Total Points: 20 Obtained Points: 20

(i) $S \rightarrow SaS | Sa | a | b$

Solution:

$$S \rightarrow aS' | bS'$$

$$S' \rightarrow aSS' | aS' | \epsilon$$

(ii) $S \rightarrow SaA | Ab | a$
 $A \rightarrow Ac | AdS | b | \epsilon$

Solution:

$$S \rightarrow AbS' | aS'$$

$$S' \rightarrow aAS' | \epsilon$$

$$A \rightarrow bA' | \epsilon$$

$$A' \rightarrow cA' | dSA' | \epsilon$$

(iii) $S \rightarrow SS- | SS* | num | id$

Solution:

$$S \rightarrow numS' | idS'$$

$$S' \rightarrow S-S' | S*S' | \epsilon$$

Q. No. 3: Consider the following grammar:
 $E \rightarrow E +$
 $T \rightarrow T -$
 $F \rightarrow F \cdot$
Construct the parse tree for the expression $E + T - F \cdot$

Solution:

Reduce = R Shift = S

Stack	input	action
\$	(id*(id+id))\$	SC (S)
\$ (	id*(id+id))\$	(C id S
\$ (id	*(id+id))\$	id → + R id
\$ (+	*(id+id))\$	(C + S
\$ (+ id	(id+id))\$	+ (C S
\$ (+ id)	id+id))\$	(C id S
\$ (+ id) +	+id))\$	id → + R id
\$ (+ id) + id	+id))\$	(C + S
\$ (+ id) + id)	id))\$	+ (C id S
\$ (+ id) + id) +	)\$	id →) R id
\$ (+ id) + id) + id	)\$	+ →) R +
\$ (+ id) + id) + id)	)\$	(=) S
\$ (+ id) + id) + id) +	)\$	) →) R)
\$ (+ id) + id) + id) + id	)\$	(=) S
\$ (+ id) + id) + id) + id)	\$	) → \$ R)
\$ (+ id) + id) + id) + id) +	\$	(→ \$ R (
\$ (+ id) + id) + id) + id) + id	\$	* → \$ R *
\$ (+ id) + id) + id) + id) + id)	\$	(→ \$ R (
\$	\$	accept

id	+	*	(	)	\$
id	e1	→	→	e1	→
+	(	→	(	(	→
*	e1	→	→	(	→
(	(	(	(	=	e1
)	e1	→	→	e1	→
\$	(	(	(	(	e3 acc

Q. No. 2: Consider the following grammar:

- $T' \rightarrow T$
 1. $T \rightarrow R$
 2. $T \rightarrow a T c$
 3. $R \rightarrow \epsilon$
 4. $R \rightarrow b R$

Its SLR parse table is given below. Show the actions of SLR parser on the input aabbbcc\$.

	Actions				Goto	
	a	b	c	\$	T	R
0	S3	S4	R3	R3	1	2
1				Accept		
2			R1	R1		
3	S3	S4	R3	R3	5	2
4		S4	R3	R3		6
5			S7			
6			R4	R4		
7			R2	R2		

Solution:

Stack	Input	Action
0	aabbbcc\$	S3
0a3	abbbcc\$	S3
0a3a3	bbcc\$	S4
0a3a3b4	bcc\$	S4
0a3a3b4b4	cc\$	S4
0a3a3b4b4b4	cc\$	R3, REDUCE by $R \rightarrow \epsilon$
0a3a3b4b4b4R6	cc\$	R4, REDUCE by $R \rightarrow bR$
0a3a3b4b4R6	cc\$	R4, REDUCE by $R \rightarrow bR$

Q. No. 4: Consider the following Operator grammar:

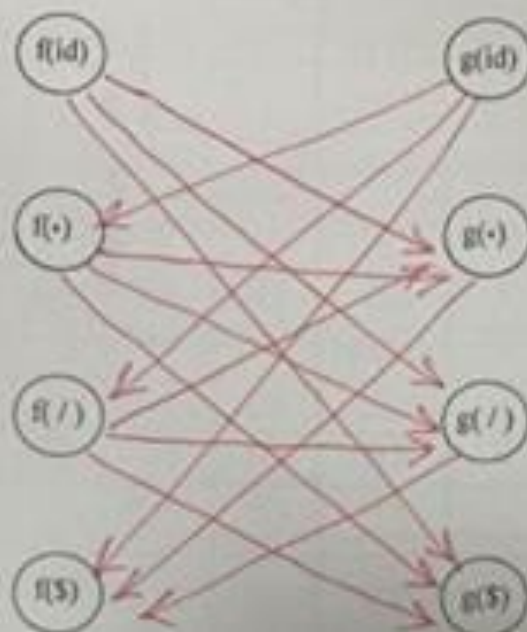
(2)

$$E \rightarrow E * E \mid E / E \mid id$$

First construct its directed graph and then construct its Operator-Precedence Functions table.

Solution:

Directed Graph



Operator-Precedence Functions table

	id	*	/	\$
f	2	2	2	0
g	3	1	1	0